

Livrable 2 – Projet Nutriscan

Plateforme analytique nutritionnelle à grande échelle



NutriScan

Auteur : DESROUSSEaux TOM

Table des matières

Livrable 2 – Projet Nutriscan	1
Table des matières	2
Définition sujet	3
Besoins utilisateurs	3
Connexion SparkSession	4
Scripts staging PySpark (couche silver)	5
6 scripts marts PySpark (gold/) + tables Trino.....	12
Tables Trino GOLD :	20
Scripts de tests qualité PySpark	22
Vues sémantiques Trino / Superset	26
Vue sem_nutriscore_moyen :	26
Vue sem_pct_produits_nutriscore_a	27
Vue sem_sel_moyen_categorie	27
Vue sem_taux_completude	28
Vue sem_nb_produits_actifs	28
6 requêtes OLAP Trino documentées	29
Requête 1 - ROLLUP catégories et pays	29
Requête 2 - Classement des marques par catégorie.....	30
Requête 3 - Cumul temporel annuel	30
Requête 4 - PIVOT Nutri-Score par catégorie.....	31
Requête 5 - SLICE France : top 10 marques par sel moyen	32
Requête 6 - Fenêtre glissante sur 3 mois	33
DAG Airflow dans Docker	35
Fonctionnement du DAG Airflow.....	35
Description des tâches du DAG	35
Gestion des dépendances	39
Rapport data quality final.....	40
1. Taux de complétude	40
2. Taux de doublons.....	41
Conclusion	43

Définition sujet

NutriScan est une startup spécialisée dans l'analyse de la qualité nutritionnelle des produits alimentaires. Sa mission : fournir des insights actionnables aux grandes surfaces, aux professionnels de santé et aux organismes publics de santé, à partir des données Open Food Facts.

Nous devons donc réaliser les missions suivantes :

- Explorer et comprendre la structure des données brutes Open Food Facts, identifier les informations pertinentes pour les besoins métier et concevoir un modèle dimensionnel adapté.
- Construire un pipeline de données de bout en bout : ingestion, stockage, transformation et mise à disposition des données pour l'analyse.
- Produire des analyses répondant aux besoins des utilisateurs NutriScan, avec une attention particulière à la performance des requêtes.
- Restituer les résultats dans un tableau de bord interactif accompagné d'un storytelling structure, adresse à des profils utilisateurs distincts.

Besoins utilisateurs

Concernant les attentes des utilisateurs, nous avons les exigences suivantes :

Profil 1 : Nutritionniste / Professionnel de santé

- Comparaison de la qualité nutritionnelle des produits par catégorie, origine géographique et période donnée
- Identification des produits et catégories les plus ou les moins favorables nutritionnellement
- Analyse des principales valeurs nutritionnelles : sucres, graisses, sel, protéines, fibres et énergie
- Évaluation du Nutri-Score et du niveau de transformation industrielle des produits
- Suivi de l'évolution de la qualité nutritionnelle des produits dans le temps

Profil 2 : Analyste marché / Grande distribution

- Identification des marques dominantes par catégorie de produits
- Comparaison du positionnement nutritionnel des marques sur une catégorie donnée
- Analyse du niveau de transformation des produits par marque et par catégorie
- Suivi de l'évolution du nombre de références et de la qualité nutritionnelle dans le temps

- Détection des tendances de marché à partir des nouveaux produits créés

Profil 3 : Utilisateur application mobile NutriScan

- Recherche rapide d'un produit par code-barres EAN
- Consultation du profil nutritionnel complet d'un produit
- Affichage du Nutri-Score, des valeurs nutritionnelles clés et du niveau de transformation
- Proposition d'alternatives mieux notées dans la même catégorie
- Comparaison simple entre un produit scanné et ses alternatives

Ces besoins spécifiques par utilisateurs doivent être pris en compte pour réaliser ce projet. On pourrait imaginer cette liste s'agrandir avec le temps pour répondre à de nouveaux besoins.

Connexion SparkSession

Pour établir la connexion entre Pyspark et minio, j'ai utilisé un objet SparkSession.

```
# transform/utils/spark_session.py
import os
if os.name == 'nt':
    os.environ['HADOOP_HOME'] = 'C:/hadoop'
    os.environ['PATH'] += ';C:/hadoop/bin'

from pyspark.sql import SparkSession

def get_spark(app_name='NutriScan'):
    minio_endpoint = os.getenv("MINIO_ENDPOINT", "http://localhost:9000")
    return SparkSession.builder \
        .appName(app_name) \
        .master('local[*]') \
        .config('spark.jars.packages',
                'org.apache.hadoop:hadoop-aws:3.3.4,'
                'com.amazonaws:aws-java-sdk-bundle:1.12.262') \
        .config('spark.sql.shuffle.partitions', '4') \
        .config('spark.hadoop.fs.s3a.endpoint', minio_endpoint) \
        .config('spark.hadoop.fs.s3a.access.key', 'minioadmin') \
        .config('spark.hadoop.fs.s3a.secret.key', 'minioadmin') \
        .config('spark.hadoop.fs.s3a.path.style.access', 'true') \
        .config('spark.hadoop.fs.s3a.connection.ssl.enabled', 'false') \
        .config('spark.driver.memory', '8g') \
        .getOrCreate()
```

C'est cet objet qui sera appelé dans chaque script, ce qui permet de définir la méthode de connexion une seule fois

Scripts staging PySpark (couche silver)

L'étape Silver permet de traiter les données en retirant les doublons normaliser /standardiser, remplacer les valeurs nulles, imputation etc.

J'ai réalisé cette partie avec Pyspark. C'est une librairie python qui permet de faire le traitement, en se connectant à MinIO pour récupérer le fichier parquet, le lire, faire le traitement et remettre la version silver dans le bon bucket au format parquet.

Les scripts suivants sont ceux qui ont été utilisés pour le silver :

stg_brands.py :

```
# transform/staging/stg_brands.py

from pyspark.sql.functions import col, split, trim, upper, lower, max as F_max
from transform.utils.spark_session import get_spark

spark = get_spark('stg_brands')
spark.sparkContext.setLogLevel('FATAL')

# Lecture depuis MinIO raw/
df = spark.read.parquet('s3a://raw/food/food_20260622.parquet')

# Nettoyage et standardisation
df_clean = df.select(
    col('code').alias('product_code'),
    upper(trim(split(col('brands'), ',').getItem(0))).alias('brand_name'),
    lower(trim(col('brands_tags').getItem(0))).alias('brand_tag')
).filter(
    col('product_code').isNotNull()
).filter(
    (col('brand_name').isNotNull()) & (col('brand_name') != '')
)

# 1 seule marque par produit
df_clean = df_clean.groupBy('product_code').agg(
    F_max('brand_name').alias('brand_name'),
    F_max('brand_tag').alias('brand_tag')
)

# Ecriture dans MinIO silver/
df_clean.write \
    .mode('overwrite') \
    .parquet('s3a://silver/stg_brands/')

print(f'stg_brands : {df_clean.count()} lignes ecrites dans silver/')
spark.stop()
```

stg.category:

```
# transform/staging/stg_category.py
from pyspark.sql.functions import col, explode, trim, regexp_replace, initcap,
max as F_max
from transform.utils.spark_session import get_spark

spark = get_spark('stg_category')
spark.sparkContext.setLogLevel('ERROR')

# Lecture depuis MinIO raw/
df = spark.read.parquet('s3a://raw/food/food_20260622.parquet')

# Extraction des catégories
df_category = df.select(
    col('code').alias('product_code'),
    col('categories_tags').getItem(0).alias('category_tag')
).withColumn(
    'category_tag',
    trim(col('category_tag'))
).filter(
    col('product_code').isNotNull()
).filter(
    (col('category_tag').isNotNull()) & (col('category_tag') != '')
)

# Création d'un category_name lisible depuis le tag
# Exemple : en:sweet-spreads -> Sweet Spreads
df_category = df_category.withColumn(
    'category_name',
    initcap(
        regexp_replace(
            regexp_replace(col('category_tag'), '^...:', ''),
            '-',
            ' '
        )
    )
)

# 1 seule catégorie par produit
df_category = df_category.groupBy('product_code').agg(
    F_max('category_tag').alias('category_tag'),
    F_max('category_name').alias('category_name')
)

# Ecriture dans MinIO silver/
```

```
df_category.write \
    .mode('overwrite') \
    .parquet('s3a://silver/stg_category/')

print(f'stg_category : {df_category.count()} lignes ecrites dans silver/')

spark.stop()
```

stg.countries :

```
# transform/staging/stg_countries.py
from pyspark.sql.functions import col, explode, split, trim, lower
from transform.utils.spark_session import get_spark

spark = get_spark('stg_countries')
spark.sparkContext.setLogLevel('ERROR')

# Lecture depuis MinIO raw/
df = spark.read.parquet('s3a://raw/food/food_20260622.parquet')

df_countries = df.select(
    col('code').alias('product_code'),
    explode(col('countries_tags')).alias('country_tag')
).withColumn('country_tag', lower(trim(col('country_tag'))))\
    .filter(col('product_code').isNotNull()) \
    .filter((col('country_tag').isNotNull()) & (col('country_tag') != '')) #
Exclure les country_tag vides ou NULL

df_countries = df_countries.dropDuplicates(['product_code', 'country_tag'])

df_countries.write.mode('overwrite').parquet('s3a://silver/stg_countries/')
print(f'stg_countries : {df_countries.count()} lignes')
spark.stop()
```

stg_date :

```
# transform/staging/stg_date.py
from pyspark.sql.functions import col, trim, to_date, from_unixtime, year,
quarter, month, dayofweek, when, max as F_max
from transform.utils.spark_session import get_spark

spark = get_spark('stg_date')
spark.sparkContext.setLogLevel('FATAL')

# Lecture depuis MinIO raw/
df = spark.read.parquet('s3a://raw/food/food_20260622.parquet')
```

```

# Conversion timestamp to date
df_clean = df.select(
    col('code').alias('product_code'),
    to_date(from_unixtime(col('last_modified_t'))).alias('full_date'))\
.filter(
    col('product_code').isNotNull())\
.filter(
    col('full_date').isNotNull())

# 1 seule date par produit : on garde la plus récente
df_clean = df_clean.groupBy('product_code').agg(
    F_max('full_date').alias('full_date')
)

df_clean = df_clean.select(
    col('product_code'),
    col('full_date'),
    year(col('full_date')).alias('year'),
    quarter(col('full_date')).alias('quarter'),
    month(col('full_date')).alias('month'),
    dayofweek(col('full_date')).alias('day_of_week'),

    when(
        dayofweek(col('full_date')).isin(1,7), True
    ).otherwise(False).alias('is_weekend')
)

# Ecriture dans MinIO silver/
df_clean.write \
    .mode('overwrite') \
    .parquet('s3a://silver/stg_date/')

print(f'stg_date : {df_clean.count()} lignes ecrites dans silver/')
spark.stop()

```

stg_nutrition :

```

# transform/staging/stg_nutrition.py
from pyspark.sql.functions import col, when, row_number
from pyspark.sql.window import Window
from transform.utils.spark_session import get_spark

spark = get_spark('stg_nutrition')
spark.sparkContext.setLogLevel('ERROR')
df = spark.read.parquet('s3a://raw/food/food_20260622.parquet')

```



```

# En PySpark, utiliser spark.sql() avec la syntaxe SQL Trino.
df.createOrReplaceTempView('food_raw')

df_nutrition = spark.sql("""
    SELECT
        code AS product_code,

        element_at(
            transform(
                filter(nutriments, n -> n.name = 'energy-kcal'),
                n -> n.`100g`
            ),
            1
        ) AS energy_kcal,

        element_at(
            transform(
                filter(nutriments, n -> n.name = 'fat'),
                n -> n.`100g`
            ),
            1
        ) AS fat_g,

        element_at(
            transform(
                filter(nutriments, n -> n.name = 'saturated-fat'),
                n -> n.`100g`
            ),
            1
        ) AS saturated_fat_g,

        element_at(
            transform(
                filter(nutriments, n -> n.name = 'sugars'),
                n -> n.`100g`
            ),
            1
        ) AS sugars_g,

        element_at(
            transform(
                filter(nutriments, n -> n.name = 'fiber'),
                n -> n.`100g`
            ),
            1
        ) AS fiber_g,

        element_at(

```

```

        transform(
            filter(nutriments, n -> n.name = 'proteins'),
            n -> n.`100g`
        ),
        1
    ) AS proteins_g,

    element_at(
        transform(
            filter(nutriments, n -> n.name = 'salt'),
            n -> n.`100g`
        ),
        1
    ) AS salt_g,

    nutriscore_score,
    to_date(from_unixtime(last_modified_t)) AS last_modified_date,
    last_modified_t

FROM food_raw
WHERE code IS NOT NULL
""")

# Garder uniquement la version la plus récente par produit
w = Window.partitionBy('product_code').orderBy(
    col('last_modified_t').desc_nulls_last()
)

df_nutrition = df_nutrition.withColumn(
    'rn',
    row_number().over(w)
).filter(
    col('rn') == 1
).drop(
    'rn',
    'last_modified_t'
)

df_nutrition.write.mode('overwrite').parquet('s3a://silver/stg_nutrition/')
print(f'stg_nutrition : {df_nutrition.count()} lignes')
spark.stop()

```

stg_products :

```

# transform/staging/stg_products.py
from pyspark.sql.functions import col, when, trim

```

```

from transform.utils.spark_session import get_spark

spark = get_spark('stg_products')
spark.sparkContext.setLogLevel('ERROR')

# Lecture depuis MinIO raw/
df = spark.read.parquet('s3a://raw/food/food_20260622.parquet')

# Nettoyage et standardisation
df_clean = df.select(
    col('code').alias('product_code'),
    col('product_name').getItem(0).getField('text').alias('product_name'),
    when(
        col('nutriscore_grade').isin('a','b','c','d','e'),
        col('nutriscore_grade')
    ).otherwise(None).alias('nutriscore_grade'),
    col('nova_group'),
    col('environmental_score_grade'),
    col('quantity'),
).filter(col('code').isNotNull())

# Ecriture dans MinIO silver/
df_clean.write \
    .mode('overwrite') \
    .parquet('s3a://silver/stg_products/')

print(f'stg_products : {df_clean.count()} lignes ecrites dans silver/')
spark.stop()

```

Les traitements réalisés permettent, entre autres, de renommer les colonnes selon la nomenclature du schéma de dimension, de filtrer les lignes sans identifiant produit, de normaliser certaines valeurs textuelles et de supprimer les doublons. Par exemple, les marques sont standardisées en majuscules afin d'éviter que des variantes comme "Coca Cola", "COCA COLA" ou "coca cola" soient considérées comme des marques différentes dans la dim_brand. Les pays sont également normalisés en minuscules pour éviter les doublons liés à la casse dans les tags (ex : « de:Allemagne » et « en:Allemagne »).

Un point important de cette étape est l'application du grain attendu pour les futures tables Gold. Les tables stg_brands, stg_category, stg_date et stg_nutrition sont ramenées à une seule ligne par produit. Cela me permet d'éviter de dédupliquer les lignes lors de la construction de la table de faits, ce que j'ai fait au début et m'a mené à avoir 13M de lignes dans ma table de fait au lieu des 4.7M attendus, mon grain était donc de produit x pays x dimension x brand x category. À l'inverse, stg_countries

conserve plusieurs lignes par produit, car le grain final de Fact_Nutrition est défini pour être en produit x pays. J'ai donc appliqué un traitement avec un groupby sur les brands /category name & tag et j'ai gardé uniquement la valeur la plus élevée à chaque fois afin d'avoir le plus d'informations sur la marque ou la catégorie en n'ayant pas de doublons dans la dimension.

Le script stg_nutrition extrait également les valeurs nutritionnelles depuis le champ nutriments, qui est stocké sous forme de tableau de structures. Les fonctions Spark permettent de récupérer les valeurs pour 100g, comme l'énergie, les graisses, les sucres, les protéines ou le sel. Lorsqu'un produit apparaît plusieurs fois dans le fichier source, seule la version la plus récente est conservée à partir du champ last_modified_t.

Enfin, les champs de date sont transformés à partir d'un timestamp Unix vers une vraie date (DD/MM/YYYY). Des informations comme l'année, le trimestre, le mois, le jour de la semaine et le is_week-end sont indiqués dans des colonnes séparées, ce qui aidera pour faire de l'analyse temporelle par la suite, et améliore la performance de la requête.

Cette étape permet donc de passer d'un fichier brut volumineux et hétérogène à un ensemble de tables de staging plus propres, structurées et alignées avec le modèle dimensionnel conçu dans le livrable 1. Ces tables servent ensuite de base pour construire les dimensions et la table de faits dans la couche Gold.

6 scripts marts PySpark (gold/) + tables Trino

La couche Gold correspond à la partie finale du pipeline. Elle permet de transformer les tables de staging de la couche Silver en tables prêtes pour l'analyse décisionnelle. L'objectif est de construire un modèle dimensionnel exploitable dans Trino et Superset, composé de dimensions et d'une table de faits, car nous sommes sur un modèle en étoile.

Dans cette étape, nous avons créé les dimensions dim_product, dim_brand, dim_category, dim_country et dim_date. Chaque dimension est construite à partir de sa table Silver correspondante. Les données sont filtrées, dédoublées, puis enrichies avec une clé de substitution générée avec row_number(). Ces surrogate key, comme product_sk, brand_sk, category_sk, country_sk ou date_sk, permettent ensuite de relier proprement la table de faits aux dimensions, afin d'obtenir toutes les informations sur chaque produits.

```
# transform/marts/dim_brand.py
from pyspark.sql.functions import col, row_number, lower, regexp_replace,
trim, first
from pyspark.sql.window import Window
from transform.utils.spark_session import get_spark
```

```

spark = get_spark('dim_brand')
spark.sparkContext.setLogLevel('ERROR')

# Lire depuis silver/
df = spark.read.parquet('s3a://silver/stg_brands/')

df_dedup = df.filter(
    col('brand_name').isNotNull()
).filter(
    col('brand_name') != ''
).groupBy(
    'brand_name'
).agg(
    first('brand_tag', ignorenulls=True).alias('brand_tag')
)

# Deduplication et surrogate key
w = Window.orderBy('brand_name')

df_dim = df_dedup.withColumn('brand_sk', row_number().over(w))

df_dim = df_dim.select(
    'brand_sk',
    'brand_name',
    'brand_tag'
)

# Ecriture dans gold/
df_dim.write.mode('overwrite').parquet('s3a://gold/dim_brand/')

print(f'dim_brand : {df_dim.count()} lignes')
spark.stop()

```

dim_category:

```

# transform/marts/dim_category.py
from pyspark.sql.functions import col, row_number
from pyspark.sql.window import Window
from transform.utils.spark_session import get_spark

spark = get_spark('dim_category')
spark.sparkContext.setLogLevel('ERROR')

# Lire depuis silver/
df = spark.read.parquet('s3a://silver/stg_category/')

# Deduplication et surrogate key

```

```

w = Window.orderBy('category_tag')

df_dim = df.select(
    'category_tag',
    'category_name'
).filter(
    col('category_tag').isNotNull()
).filter(
    col('category_tag') != ''
).dropDuplicates(['category_tag']) \
    .withColumn('category_sk', row_number().over(w))

# Remettre les colonnes dans l'ordre attendu
df_dim = df_dim.select(
    'category_sk',
    'category_tag',
    'category_name'
)

# Ecriture dans gold/
df_dim.write.mode('overwrite').parquet('s3a://gold/dim_category/')

print(f'dim_category : {df_dim.count()} lignes')

spark.stop()

```

dim_country

```

# transform/marts/dim_country.py
from pyspark.sql.functions import col, row_number, regexp_replace, initcap
from pyspark.sql.window import Window
from transform.utils.spark_session import get_spark

spark = get_spark('dim_country')
spark.sparkContext.setLogLevel('ERROR')

# Lire depuis silver/
df = spark.read.parquet('s3a://silver/stg_countries/')

# Deduplication et surrogate key
w = Window.orderBy('country_tag')

df_dim = df.select(
    col('country_tag')
).filter(
    col('country_tag').isNotNull()
).filter(

```

```

        col('country_tag') != ''
    ).dropDuplicates(['country_tag']) \
    .withColumn(
        'country_name',
        initcap(regexp_replace(regexp_replace(col('country_tag'), '^...:', ''), '- ', ' ')))
    ).withColumn(
        'country_sk',
        row_number().over(w)
    )

df_dim = df_dim.select(
    'country_sk',
    'country_tag',
    'country_name'
)

# Ecriture dans gold/
df_dim.write.mode('overwrite').parquet('s3a://gold/dim_country/')

print(f'dim_country : {df_dim.count()} lignes')
spark.stop()

```

dim_date

```

# transform/marts/dim_date.py
from pyspark.sql.functions import col, row_number
from pyspark.sql.window import Window
from transform.utils.spark_session import get_spark

spark = get_spark('dim_date')
spark.sparkContext.setLogLevel('ERROR')

# Lire depuis silver/
df = spark.read.parquet('s3a://silver/stg_date/')

# Deduplication et surrogate key
w = Window.orderBy('full_date')

df_dim = df.select(
    'full_date',
    'year',
    'quarter',
    'month',
    'day_of_week',
    'is_weekend'
).filter(

```

```

        col('full_date').isNull()
    ).dropDuplicates(['full_date']) \
    .withColumn(
        'date_sk',
        row_number().over(w)
    )

df_dim = df_dim.select(
    'date_sk',
    'full_date',
    'year',
    'quarter',
    'month',
    'day_of_week',
    'is_weekend'
)

# Ecriture dans gold/
df_dim.write.mode('overwrite').parquet('s3a://gold/dim_date/')

print(f'dim_date : {df_dim.count()} lignes')
spark.stop()

```

dim_product

```

# transform/marts/dim_product.py
from pyspark.sql.functions import col, row_number
from pyspark.sql.window import Window
from transform.utils.spark_session import get_spark

spark = get_spark('dim_product')

# Lire depuis silver/
df = spark.read.parquet('s3a://silver/stg_products/')

# Deduplication et surrogate key
w = Window.orderBy('product_code')
df_dim = df.select(
    'product_code', 'product_name', 'nutriscore_grade',
    'nova_group', 'environmental_score_grade', 'quantity'
).dropDuplicates(['product_code']) \
.withColumn('product_sk', row_number().over(w))

# Ecriture dans gold/
df_dim.write.mode('overwrite').parquet('s3a://gold/dim_product/')

print(f'dim_product : {df_dim.count()} lignes')

```



```
spark.stop()
```

fact_nutrition

```
# transform/marts/fact_nutrition.py
from pyspark.sql.functions import col, row_number
from pyspark.sql.window import Window
from transform.utils.spark_session import get_spark

spark = get_spark('fact_nutrition')

# Lecture des sources
nutrition = spark.read.parquet('s3a://silver/stg_nutrition/')
countries = spark.read.parquet('s3a://silver/stg_countries/')
categories = spark.read.parquet('s3a://silver/stg_category/')
brands = spark.read.parquet('s3a://silver/stg_brands/')
dates = spark.read.parquet('s3a://silver/stg_date/')
dim_prod = spark.read.parquet('s3a://gold/dim_product/')
dim_cat = spark.read.parquet('s3a://gold/dim_category/')
dim_brand = spark.read.parquet('s3a://gold/dim_brand/')
dim_country = spark.read.parquet('s3a://gold/dim_country/')
dim_date = spark.read.parquet('s3a://gold/dim_date/')

# Construction de la table de faits
fact = nutrition \
    .join(countries, 'product_code', 'inner') \
    .join(dim_prod, 'product_code', 'left') \
    .join(categories, 'product_code', 'left') \
    .join(dim_cat, 'category_tag', 'left') \
    .join(brands, 'product_code', 'left') \
    .join(dim_brand, 'brand_name', 'left') \
    .join(dim_country, 'country_tag', 'left') \
    .join(dates, 'product_code', 'left') \
    .join(dim_date, col('last_modified_date') == dim_date['full_date'],
'left') \
    .select( #AJOUT ICI
        dim_prod['product_sk'],
        dim_cat['category_sk'],
        dim_brand['brand_sk'],
        dim_country['country_sk'],
        dim_date['date_sk'],
        col('energy_kcal'),
        col('fat_g'),
        col('salt_g'),
        col('nutriscore_score'), #AJOUT ICI
        col('saturated_fat_g'),
```

```

        col('sugars_g'),
        col('fiber_g'),
        col('proteins_g')
    )

# Supprime les doublons sur les colonnes de la table de faits à cause de
# jointures multiples
fact = fact.dropDuplicates([
    'product_sk',
    'category_sk',
    'brand_sk',
    'country_sk',
    'date_sk',
    'energy_kcal',
    'fat_g',
    'salt_g',
    'nutriscore_score',
    'saturated_fat_g',
    'sugars_g',
    'fiber_g',
    'proteins_g'
])

# Ajout d'une clé technique pour la table de faits
w = Window.orderBy(
    'product_sk',
    'category_sk',
    'brand_sk',
    'country_sk',
    'date_sk'
)
fact = fact.withColumn(
    'nutrition_fact_sk',
    row_number().over(w)
)

# Remettre les colonnes dans l'ordre final
fact = fact.select(
    'nutrition_fact_sk',

    'product_sk',
    'category_sk',
    'brand_sk',
    'country_sk',
    'date_sk',

    'energy_kcal',

```

```

    'fat_g',
    'salt_g',
    'nutriscore_score',
    'saturated_fat_g',
    'sugars_g',
    'fiber_g',
    'proteins_g'
)

fact.write.mode('overwrite').parquet('s3a://gold/fact_nutrition/')
print(f'fact_nutrition : {fact.count()} lignes')
spark.stop()

```

Les scripts Gold appliquent également une logique de déduplication afin d'éviter plusieurs lignes pour une même entité métier. C'est purement sécuritaire car la table silver est déjà sensée être dédoublonnée. Par exemple, dim_product est dédoublonnée sur product_code, dim_category sur category_tag, dim_country sur country_tag, et dim_brand sur brand_name. Cela permet d'obtenir des dimensions plus propres, avec des données uniques et permet de simplifier l'analyse par la suite.

La table fact_nutrition est construite en croisant les mesures nutritionnelles avec les différentes dimensions. Les tables de staging sont utilisées comme tables intermédiaires pour retrouver les informations métier associées à chaque produit, comme le pays, la marque principale ou la catégorie principale. Une fois les jointures effectuées, la table de faits ne conserve que les clés techniques des dimensions et les mesures nutritionnelles. Les jointures sont conçues pour créer des doublons car on insère potentiellement 2 fois les données. J'ajoute donc une étape de dropduplicate pour s'assurer qu'une seule version des données est gardée au cas où.

Le grain retenu pour fact_nutrition est un produit pour un pays donné. Cela signifie qu'un même produit peut apparaître plusieurs fois s'il est vendu dans plusieurs pays, mais les catégories, marques et dates sont ramenées à une seule valeur principale par produit afin d'éviter une multiplication artificielle des lignes (13M lignes au lieu de 4.7M).

Une clé primaire nutrition_fact_sk est également générée pour identifier simplement chaque ligne de la table de faits.

Enfin, les résultats (dimensions et table de fait) sont écrits dans le bucket gold/ de MinIO au format Parquet. Ces fichiers sont ensuite utilisés dans Trino sous forme de tables externes afin d'être interrogeables depuis un moteur de recherche SQL comme Superset pour vérifier les données insérées. Cette couche Gold constitue donc la partie finale et analytique du pipeline NutriScan.

Tables Trino GOLD :

```
-- Créer le schéma gold
CREATE SCHEMA IF NOT EXISTS hive.nutriscan_gold
WITH (location = 's3a://gold/');

-- Dim_Product
DROP TABLE IF EXISTS hive.nutriscan_gold.dim_product;
CREATE TABLE IF NOT EXISTS hive.nutriscan_gold.dim_product (
    product_sk      INTEGER,
    product_code    VARCHAR,
    product_name    VARCHAR,
    nutriscore_grade VARCHAR,
    nova_group      INTEGER,
    environmental_score_grade VARCHAR,
    quantity        VARCHAR
)
WITH (external_location = 's3a://gold/dim_product/', format = 'PARQUET')

-- Dim_Category
DROP TABLE IF EXISTS hive.nutriscan_gold.dim_category;

CREATE TABLE IF NOT EXISTS hive.nutriscan_gold.dim_category (
    category_sk INTEGER,
    category_tag VARCHAR,
    category_name VARCHAR
)
WITH (
    external_location = 's3a://gold/dim_category/',
    format = 'PARQUET'
);

-- Dim_Country
DROP TABLE IF EXISTS hive.nutriscan_gold.dim_country;

CREATE TABLE IF NOT EXISTS hive.nutriscan_gold.dim_country (
    country_sk INTEGER,
    country_tag VARCHAR,
    country_name VARCHAR
)
WITH (
    external_location = 's3a://gold/dim_country/',
    format = 'PARQUET'
);

-- Dim_Brand
DROP TABLE IF EXISTS hive.nutriscan_gold.dim_brand;
CREATE TABLE IF NOT EXISTS hive.nutriscan_gold.dim_brand (
```

```

    brand_sk INTEGER,
    brand_name VARCHAR,
    brand_tag VARCHAR
)
WITH (
    external_location = 's3a://gold/dim_brand/',
    format = 'PARQUET'
);

-- Dim_Date
DROP TABLE IF EXISTS hive.nutriscan_gold.dim_date;

CREATE TABLE IF NOT EXISTS hive.nutriscan_gold.dim_date (
    date_sk INTEGER,
    full_date DATE,
    year INTEGER,
    quarter INTEGER,
    month INTEGER,
    day_of_week INTEGER,
    is_weekend BOOLEAN
)
WITH (
    external_location = 's3a://gold/dim_date/',
    format = 'PARQUET'
);

-- Fact_Nutrition (grain : un produit x un pays)
DROP TABLE IF EXISTS hive.nutriscan_gold.fact_nutrition;

CREATE TABLE IF NOT EXISTS hive.nutriscan_gold.fact_nutrition (
    nutrition_fact_sk INTEGER,
    product_sk          INTEGER,
    category_sk         INTEGER,
    brand_sk            INTEGER,
    country_sk          INTEGER,
    date_sk             INTEGER,

    energy_kcal         REAL,
    fat_g               REAL,
    salt_g              REAL,
    nutriscore_score    INTEGER,
    saturated_fat_g     REAL,
    sugars_g            REAL,
    fiber_g             REAL,
    proteins_g          REAL
)
WITH (
    external_location = 's3a://gold/fact_nutrition/',
    format = 'PARQUET'
);

```

```
);

-- Verification finale
SHOW TABLES FROM hive.nutriscan_gold;
SELECT COUNT(*) FROM hive.nutriscan_gold.dim_product;

SHOW TABLES FROM hive.nutriscan_gold;
SELECT COUNT(*) FROM hive.nutriscan_gold.fact_nutrition;
-- Doit retourner 0
```

Les tables Trino de la couche Gold permettent d'exposer les fichiers Parquet générés par les scripts PySpark sous forme de tables SQL interrogeables dans Superset notamment mais dans les dashboards par la suite. Les données restent physiquement stockées dans MinIO, dans le bucket gold/.

J'ai d'abord créé le schéma hive.nutriscan_gold, associé à l'emplacement minio « s3a://gold/ ». Ensuite, chaque table Gold est déclarée avec son schéma SQL et son chemin Parquet correspondant. Par exemple, la table dim_product pointe vers « s3a://gold/dim_product/ », tandis que fact_nutrition pointe vers « s3a://gold/fact_nutrition/ ».

Les tables de dimensions (dim_product, dim_category, dim_country, dim_brand, dim_date) contiennent les informations complémentaires pour chaque produit. Les clés surrogates (sk) permettent de faire les jointures avec la table de faits et de faire des requêtes plus complexes afin de réaliser une analyse avancée et de répondre au besoin métier.

La table fact_nutrition contient les mesures nutritionnelles principales, comme l'énergie, les graisses, les sucres, le sel, les protéines, les fibres ou encore le score nutritionnel. Son grain est : produit x pays. Elle conserve également les clés nécessaires pour relier les produits aux dimensions.

Ces tables Trino Gold peuvent être utilisées dans Superset pour faire des requêtes semblables au SQL. C'est grâce à ces tables que je peux faire les requêtes OLAP et la visualisation des données dans Superset. Elles permettent d'analyser les données par produit, marque, catégorie, pays ou date en un temps raisonnable grâce au nettoyage et à la présence de données nécessaires uniquement. Les commandes SHOW TABLES et COUNT(*) permettent enfin de vérifier que les tables ont bien été créées et qu'elles pointent correctement vers les données générées dans MinIO.

Scripts de tests qualité PySpark

Afin de vérifier la cohérence des données produites en couche Gold, j'ai ajouté deux scripts de tests qualité PySpark. Ces tests permettent de contrôler automatiquement

certains points essentiels avant l'exploitation des données et la réalisation des dashboards.

Test_fact_nutrition:

```
# transform/tests/test_fact_nutrition.py

from transform.utils.spark_session import get_spark

spark = get_spark('test_fact_nutrition')
spark.sparkContext.setLogLevel('OFF')

df = spark.read.parquet('s3a://gold/fact_nutrition/')

errors = []
success = []

# Test 1 : product_sk non NULL
null_product_sk = df.filter(df.product_sk.isNull()).count()
if null_product_sk > 0:
    errors.append(f'ECHEC not_null product_sk : {null_product_sk} NULL')
else:
    success.append('OK not_null product_sk')

# Test 2 : category_sk non NULL
null_category_sk = df.filter(df.category_sk.isNull()).count()
if null_category_sk > 0:
    errors.append(f'ECHEC not_null category_sk : {null_category_sk} NULL')
else:
    success.append('OK not_null category_sk')

# Test 3 : brand_sk non NULL
null_brand_sk = df.filter(df.brand_sk.isNull()).count()
if null_brand_sk > 0:
    errors.append(f'ECHEC not_null brand_sk : {null_brand_sk} NULL')
else:
    success.append('OK not_null brand_sk')

# Test 4 : country_sk non NULL
null_country_sk = df.filter(df.country_sk.isNull()).count()
if null_country_sk > 0:
    errors.append(f'ECHEC not_null country_sk : {null_country_sk} NULL')
else:
    success.append('OK not_null country_sk')

# Test 5 : date_sk non NULL
null_date_sk = df.filter(df.date_sk.isNull()).count()
if null_date_sk > 0:
    errors.append(f'ECHEC not_null date_sk : {null_date_sk} NULL')
```

```

else:
    success.append('OK not_null date_sk')

# Test 6 : energy_kcal positive ou nulle
invalid_energy = df.filter(
    df.energy_kcal.isNotNull() & (df.energy_kcal < 0)
).count()

if invalid_energy > 0:
    errors.append(f'ECHEC positive_values energy_kcal : {invalid_energy} valeurs negatives')
else:
    success.append('OK positive_values energy_kcal')

# Test 7 : salt_g positif ou nul
invalid_salt = df.filter(
    df.salt_g.isNotNull() & (df.salt_g < 0)
).count()

if invalid_salt > 0:
    errors.append(f'ECHEC positive_values salt_g : {invalid_salt} valeurs negatives')
else:
    success.append('OK positive_values salt_g')

# Test 8 : nutriscore_score entre -15 et 40
invalid_nutriscore_score = df.filter(
    df.nutriscore_score.isNotNull() &
    (
        (df.nutriscore_score < -15) |
        (df.nutriscore_score > 40)
    )
).count()

if invalid_nutriscore_score > 0:
    errors.append(
        f'ECHEC accepted_range nutriscore_score [-15, 40] : {invalid_nutriscore_score} invalides'
    )
else:
    success.append('OK accepted_range nutriscore_score [-15, 40]')

# Résultat des tests
print('--- TESTS REUSSIS ---')
for s in success:
    print(s)

print('--- TESTS EN ECHEC ---')

```



```

if errors:
    for e in errors:
        print(e)
else:
    print('Aucun test en echec')

spark.stop()

```

test_dim_product

```

# transform/tests/test_dim_product.py
from transform.utils.spark_session import get_spark

spark = get_spark('test_dim_product')
df = spark.read.parquet('s3a://gold/dim_product/')
spark.sparkContext.setLogLevel('OFF')
errors = []

# Test 1 : product_sk non NULL
null_sk = df.filter(df.product_sk.isNull()).count()
if null_sk > 0:
    errors.append(f'ECHEC not_null product_sk : {null_sk} NULL')

# Test 2 : product_sk unique
total = df.count()
distinct = df.select('product_sk').distinct().count()
if total != distinct:
    errors.append(f'ECHEC unique product_sk : {total - distinct} doublons')

# Test 3 : product_code non NULL
null_code = df.filter(df.product_code.isNull()).count()
if null_code > 0:
    errors.append(f'ECHEC not_null product_code : {null_code} NULL')

# Test 4 : nutriscore_grade valeurs acceptees
invalid = df.filter(df.nutriscore_grade.isNotNull() &
    ~df.nutriscore_grade.isin('a','b','c','d','e')).count()

if invalid > 0:
    errors.append(f'ECHEC accepted_values nutriscore_grade : {invalid}
invalides')

if errors:
    for e in errors: print(e)
else:
    print('Tous les tests passent pour dim_product')
spark.stop()

```

Le script `test_dim_product.py` vérifie la qualité de la dimension produit. Il contrôle notamment que la clé technique `product_sk` n'est pas nulle, qu'elle est unique, que chaque produit possède bien un `product_code`, et que les valeurs du champ `nutriscore_grade` appartiennent uniquement aux valeurs attendues : a, b, c, d ou e.

Le script `test_fact_nutrition.py` contrôle la table de faits `fact_nutrition`. Il vérifie que les clés étrangères (`product_sk`, `category_sk`, `brand_sk`, `country_sk`, `date_sk`) ne sont pas nulles, afin de garantir que la table de faits soit bien reliée aux dimensions et qu'il n'y ait pas d'erreur par la suite. Il contrôle aussi que certaines mesures nutritionnelles restent cohérentes, par exemple que `energy_kcal` et `salt_g` ne contiennent pas de valeurs négatives, et que `nutriscore_score` reste dans l'intervalle attendu entre -15 et 40, sinon valeur aberrante.

Ces scripts affichent les tests réussis et les tests en échec, ce qui permet d'identifier rapidement les éventuels problèmes de qualité après l'exécution du pipeline. Ils constituent donc une première couche de validation automatique avant l'analyse des données.

Vues sémantiques Trino / Superset

Les vues sémantiques permettent de préparer des requêtes prêtes à l'emploi pour l'analyse dans Superset.

Au lieu de réécrire à chaque fois des jointures entre la table de faits et les dimensions, nous créons des vues SQL qui regroupent déjà les indicateurs importants.

Elles servent donc à simplifier l'exploitation des données par les utilisateurs ou dans les dashboards. Les données restent stockées dans les tables Gold, mais les vues permettent d'avoir une couche plus lisible et orientée analyse.

Dans notre cas, les vues créées permettent de répondre à plusieurs besoins métier :

Vue `sem_nutriscore_moyen` :

Cette vue calcule le Nutri-Score moyen par catégorie, marque, pays, année et mois. Elle permet donc d'analyser la qualité nutritionnelle moyenne des produits selon plusieurs axes d'analyse.

```
-- Vue 1 : nutriscore moyen (executer dans Superset SQL Lab)
CREATE OR REPLACE VIEW hive.nutriscan_gold.sem_nutriscore_moyen AS
SELECT
    dc.category_name,
    db.brand_name,
    dco.country_name,
    dd.year,
```

```

    dd.month,
    ROUND(AVG(fn.nutriscore_score), 2) AS nutriscore_moyen,
    COUNT(DISTINCT fn.product_sk) AS nb_produits
FROM hive.nutriscan_gold.fact_nutrition fn
JOIN hive.nutriscan_gold.dim_category dc ON fn.category_sk = dc.category_sk
JOIN hive.nutriscan_gold.dim_brand db ON fn.brand_sk = db.brand_sk
JOIN hive.nutriscan_gold.dim_country dco ON fn.country_sk = dco.country_sk
JOIN hive.nutriscan_gold.dim_date dd ON fn.date_sk = dd.date_sk
WHERE fn.nutriscore_score IS NOT NULL
GROUP BY dc.category_name, db.brand_name, dco.country_name, dd.year, dd.month;

```

Vue sem_pct_produits_nutriscore_a

Cette vue calcule, pour chaque marque, le pourcentage de produits ayant un Nutri-Score A. Elle permet d'identifier les marques dont une part importante des produits est bien notée nutritionnellement. C'est utile pour comparer les marques entre elles sur un critère simple et facilement compréhensible.

```

-- Vue 2 : Pourcentage de produits Nutri-Score A par marque

CREATE OR REPLACE VIEW hive.nutriscan_gold.sem_pct_produits_nutriscore_a AS
SELECT
    db.brand_name,
    COUNT(DISTINCT fn.product_sk) AS nb_produits_total,
    COUNT(DISTINCT CASE
        WHEN LOWER(dp.nutriscore_grade) = 'a' THEN fn.product_sk
    END) AS nb_produits_nutriscore_a,
    ROUND(
        100.0 * COUNT(DISTINCT CASE
            WHEN LOWER(dp.nutriscore_grade) = 'a' THEN fn.product_sk
        END) / NULLIF(COUNT(DISTINCT fn.product_sk), 0),
        2
    ) AS pct_produits_nutriscore_a
FROM hive.nutriscan_gold.fact_nutrition fn
JOIN hive.nutriscan_gold.dim_product dp
    ON fn.product_sk = dp.product_sk
JOIN hive.nutriscan_gold.dim_brand db
    ON fn.brand_sk = db.brand_sk
GROUP BY db.brand_name;

```

Vue sem_sel_moyen_categorie

Cette vue calcule le sel moyen par catégorie de produit.

Elle permet de repérer les catégories qui contiennent en moyenne le plus de sel.

Cela peut être utilisé pour construire un graphique de type classement ou bar chart, afin d'identifier les catégories à surveiller.

```
-- Vue 3 : Sel moyen par catégorie
CREATE OR REPLACE VIEW hive.nutriscan_gold.sem_sel_moyen_categorie AS
WITH produit_categorie AS (
    SELECT DISTINCT
        fn.product_sk,
        fn.category_sk,
        fn.salt_g
    FROM hive.nutriscan_gold.fact_nutrition fn
    WHERE fn.salt_g IS NOT NULL
)
SELECT
    dc.category_name,
    ROUND(AVG(pc.salt_g), 3) AS sel_moyen_g,
    COUNT(DISTINCT pc.product_sk) AS nb_produits
FROM produit_categorie pc
JOIN hive.nutriscan_gold.dim_category dc
    ON pc.category_sk = dc.category_sk
GROUP BY dc.category_name;
```

Vue sem_taux_completude

Cette vue mesure le taux de complétude du champ nutriscore_score dans la table de faits. Elle permet de savoir quelle proportion des lignes possède un score nutritionnel exploitable. C'est important, car une analyse sur le Nutri-Score n'est fiable que si le champ est suffisamment renseigné.

```
-- Vue 4 – Taux de complétude du Nutri-Score
CREATE OR REPLACE VIEW hive.nutriscan_gold.sem_taux_completude AS
SELECT
    COUNT(*) AS nb_lignes_total,
    COUNT_IF(nutriscore_score IS NOT NULL) AS
nb_lignes_nutriscore_score_renseigne,
    COUNT_IF(nutriscore_score IS NULL) AS nb_lignes_nutriscore_score_null,
    ROUND(
        100.0 * COUNT_IF(nutriscore_score IS NOT NULL) / COUNT(*),
        2
    ) AS taux_completude_nutriscore_score
FROM hive.nutriscan_gold.fact_nutrition;
```

Vue sem_nb_produits_actifs

Cette vue calcule le nombre total de produits distincts présents dans la table de faits. Elle sert d'indicateur global pour connaître le volume de produits réellement exploitables dans la couche Gold.

```
-- Vue 5- Nombre total de produits actifs
CREATE OR REPLACE VIEW hive.nutriscan_gold.sem_nb_produits_actifs AS
SELECT
```

```

COUNT(DISTINCT product_sk) AS nb_produits_actifs
FROM hive.nutriscan_gold.fact_nutrition
WHERE product_sk IS NOT NULL;

select * from sem_nutriscore_moyen;
SELECT * FROM hive.nutriscan_gold.sem_pct_produits_nutriscore_a;
SELECT * FROM hive.nutriscan_gold.sem_sel_moyen_categorie;
SELECT * FROM hive.nutriscan_gold.sem_taux_completude;
SELECT * FROM hive.nutriscan_gold.sem_nb_produits_actifs;

```

En résumé, ces vues sémantiques permettent de rendre la couche Gold plus facile à utiliser dans Superset. Elles masquent la complexité des jointures entre fact_nutrition et les dimensions, tout en fournissant directement des indicateurs prêts pour les tableaux de bord.

6 requêtes OLAP Trino documentées

Les requêtes OLAP permettent d'exploiter le modèle Gold afin de réaliser des analyses multidimensionnelles. Elles s'appuient sur la table de faits fact_nutrition et sur les dimensions dim_product, dim_category, dim_brand, dim_country et dim_date. L'objectif est de montrer que le modèle permet d'analyser les données alimentaires selon plusieurs axes : catégorie, pays, marque, date et score nutritionnel. Ces requêtes peuvent ensuite être utilisées dans Superset pour produire des tableaux, classements ou graphiques.

Requête 1 - ROLLUP catégories et pays

La première requête utilise un ROLLUP sur les dimensions category_name et country_name. Elle permet de calculer le Nutri-Score moyen à plusieurs niveaux d'agrégation :

- Par catégorie et pays ;
- Par catégorie tous pays confondus ;
- Globalement, toutes catégories et tous pays confondus.

Cette requête est utile pour comparer la qualité nutritionnelle moyenne des produits selon leur catégorie et leur pays de commercialisation. Le ROLLUP permet aussi d'obtenir des totaux intermédiaires sans écrire plusieurs requêtes séparées.

```

-- Requete 1 : ROLLUP categories et pays (fournie)
SELECT
    dc.category_name,
    dco.country_name,
    ROUND(AVG(fn.nutriscore_score), 2) AS nutriscore_moyen,
    COUNT(DISTINCT fn.product_sk) AS nb_produits
FROM hive.nutriscan_gold.fact_nutrition fn
JOIN hive.nutriscan_gold.dim_category dc ON fn.category_sk = dc.category_sk

```

```
JOIN hive.nutriscan_gold.dim_country dco ON fn.country_sk = dco.country_sk
GROUP BY ROLLUP(dc.category_name, dco.country_name)
ORDER BY "nutriscore_moyen" DESC, dc.category_name NULLS LAST,
dco.country_name NULLS LAST
limit 10;
```

Requête 2 - Classement des marques par catégorie

La deuxième requête utilise la fonction RANK() qui permet de classer les marques de chaque catégorie selon leur Nutri-Score moyen.

Elle permet d'identifier les trois marques ayant le meilleur score nutritionnel moyen au sein de chaque catégorie. Cette analyse est intéressante pour comparer les marques entre elles dans un même segment de produits, par exemple les boissons, les snacks ou les produits laitiers.

```
-- Requete 2 : RANK des marques par nutriscore (fournie)
WITH ranked AS (
SELECT
    db.brand_name,
    dc.category_name,
    ROUND(AVG(fn.nutriscore_score), 2) AS
nutriscore_moyen,
    RANK() OVER (PARTITION BY dc.category_name
ORDER BY AVG(fn.nutriscore_score) ASC) AS rang
FROM hive.nutriscan_gold.fact_nutrition fn
JOIN hive.nutriscan_gold.dim_brand db ON fn.brand_sk = db.brand_sk
JOIN hive.nutriscan_gold.dim_category dc ON fn.category_sk = dc.category_sk
GROUP BY db.brand_name, dc.category_name
)
SELECT * FROM ranked WHERE rang <= 3
ORDER BY "nutriscore_moyen" ASC, category_name, rang;
```

Requête 3 - Cumul temporel annuel

La troisième requête analyse l'évolution du nombre de produits dans le temps. Elle calcule le nombre de produits distincts par mois, puis applique une somme cumulée sur l'année.

Cette requête permet de suivre l'augmentation progressive du volume de produits présents dans la base au fil des mois. Elle exploite la dimension dim_date et montre l'intérêt d'avoir préparé une dimension temporelle dans le modèle Gold.

```
-- Requete 3 : Cumul temporel annuel (fournie)
SELECT
    dd.year,
    dd.month,
    COUNT(DISTINCT fn.product_sk) AS
nb_produits_mois,
```

```

SUM(COUNT(DISTINCT fn.product_sk)) OVER (
    PARTITION BY dd.year
    ORDER BY dd.month
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
nb_produits_cumule
FROM hive.nutriscan_gold.fact_nutrition fn
JOIN hive.nutriscan_gold.dim_date dd ON fn.date_sk = dd.date_sk
GROUP BY dd.year, dd.month
ORDER BY dd.year, dd.month;

```

Requête 4 - PIVOT Nutri-Score par catégorie

La quatrième requête produit un tableau croisé par catégorie. Elle compte, pour chaque catégorie, le nombre de produits ayant un Nutri-Score a, b, c, d ou e.

Le pivot est réalisé avec des CASE WHEN, ce qui permet d'obtenir une colonne par grade Nutri-Score. Cette requête donne une vision claire de la répartition nutritionnelle des produits dans chaque catégorie. Elle permet par exemple de voir quelles catégories contiennent le plus de produits bien notés ou mal notés.

Une requête complémentaire permet aussi d'identifier le grade Nutri-Score le plus fréquent dans l'ensemble du jeu de données.

```

-- REQUETE 4 : PIVOT Nutri-Score par categorie
WITH produits_categories AS (
    SELECT DISTINCT
        fn.product_sk,
        fn.category_sk,
        LOWER(dp.nutriscore_grade) AS nutriscore_grade
    FROM hive.nutriscan_gold.fact_nutrition fn
    JOIN hive.nutriscan_gold.dim_product dp
        ON fn.product_sk = dp.product_sk
    WHERE dp.nutriscore_grade IS NOT NULL
)

SELECT
    dc.category_name,
    COUNT(DISTINCT CASE WHEN pc.nutriscore_grade = 'a' THEN pc.product_sk END)
AS nb_grade_a,
    COUNT(DISTINCT CASE WHEN pc.nutriscore_grade = 'b' THEN pc.product_sk END)
AS nb_grade_b,
    COUNT(DISTINCT CASE WHEN pc.nutriscore_grade = 'c' THEN pc.product_sk END)
AS nb_grade_c,
    COUNT(DISTINCT CASE WHEN pc.nutriscore_grade = 'd' THEN pc.product_sk END)
AS nb_grade_d,
    COUNT(DISTINCT CASE WHEN pc.nutriscore_grade = 'e' THEN pc.product_sk END)
AS nb_grade_e,
    COUNT(DISTINCT pc.product_sk) AS nb_produits_total

```

```

FROM produits_categories pc
JOIN hive.nutriscan_gold.dim_category dc
    ON pc.category_sk = dc.category_sk
GROUP BY dc.category_name
ORDER BY nb_produits_total DESC
LIMIT 50;

WITH produits_grades AS (
    SELECT DISTINCT
        fn.product_sk,
        LOWER(dp.nutriscore_grade) AS nutriscore_grade
    FROM hive.nutriscan_gold.fact_nutrition fn
    JOIN hive.nutriscan_gold.dim_product dp
        ON fn.product_sk = dp.product_sk
    WHERE dp.nutriscore_grade IS NOT NULL
)
SELECT
    nutriscore_grade,
    COUNT(DISTINCT product_sk) AS nb_produits
FROM produits_grades
GROUP BY nutriscore_grade
ORDER BY nb_produits DESC
LIMIT 1;

```

Requête 5 - SLICE France : top 10 marques par sel moyen

La cinquième requête applique un filtre sur un pays précis : la France. C'est ce qu'on appelle une opération de type slice, car on isole une partie du cube analytique selon une dimension.

La requête calcule ensuite les dix marques ayant la moyenne de sel la plus élevée parmi les produits vendus en France. Elle utilise les dimensions dim_country et dim_brand, ainsi que la mesure salt_g de la table de faits.

Cette analyse peut être utile pour repérer les marques dont les produits sont en moyenne les plus salés sur un marché donné.

```

-- REQUETE 5 : SLICE France, top 10 marques par sel moyen
WITH produits_france AS (
    SELECT DISTINCT
        fn.product_sk,
        fn.brand_sk,
        fn.salt_g
    FROM hive.nutriscan_gold.fact_nutrition fn
    JOIN hive.nutriscan_gold.dim_country dco
        ON fn.country_sk = dco.country_sk
    WHERE dco.country_tag = 'en:france'
        AND fn.salt_g IS NOT NULL
)

```



```

        AND fn.brand_sk IS NOT NULL
    )
SELECT
    db.brand_name,
    ROUND(AVG(pf.salt_g), 3) AS sel_moyen_g,
    COUNT(DISTINCT pf.product_sk) AS nb_produits
FROM produits_france pf
JOIN hive.nutriscan_gold.dim_brand db
    ON pf.brand_sk = db.brand_sk
GROUP BY db.brand_name
ORDER BY sel_moyen_g DESC
LIMIT 10;

```

Requête 6 - Fenêtre glissante sur 3 mois

La sixième requête calcule le Nutri-Score moyen par mois, puis applique une moyenne mobile sur trois mois glissants.

Cette requête permet de lisser les variations mensuelles et de mieux observer les tendances dans le temps.

```

-- REQUETE 6 : Fenetre glissante 3 mois
WITH mensuel AS (
    SELECT
        dd.year,
        dd.month,
        CONCAT(CAST(dd.year AS VARCHAR), '-', LPAD(CAST(dd.month AS VARCHAR),
2, '0')) AS periode,
        ROUND(AVG(fn.nutriscore_score), 2) AS nutriscore_moyen_mois,
        COUNT(DISTINCT fn.product_sk) AS nb_produits
    FROM hive.nutriscan_gold.fact_nutrition fn
    JOIN hive.nutriscan_gold.dim_date dd
        ON fn.date_sk = dd.date_sk
    WHERE fn.nutriscore_score IS NOT NULL
    GROUP BY dd.year, dd.month
)
SELECT
    periode,
    year,
    month,
    nutriscore_moyen_mois,
    nb_produits,

    ROUND(
        AVG(nutriscore_moyen_mois) OVER (
            ORDER BY year, month
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ),
        2

```

```

        ) AS moyenne_mobile_3_mois
FROM mensuel
ORDER BY year, month;

-- résultat attendu requete 6
WITH mensuel AS (
    SELECT
        dd.year,
        dd.month,
        CONCAT(CAST(dd.year AS VARCHAR), '-', LPAD(CAST(dd.month AS VARCHAR),
2, '0')) AS periode,
        ROUND(AVG(fn.nutriscore_score), 2) AS nutriscore_moyen_mois,
        COUNT(DISTINCT fn.product_sk) AS nb_produits
    FROM hive.nutriscan_gold.fact_nutrition fn
    JOIN hive.nutriscan_gold.dim_date dd
        ON fn.date_sk = dd.date_sk
    WHERE fn.nutriscore_score IS NOT NULL
    GROUP BY dd.year, dd.month
),
mensuel_mobile AS (
    SELECT
        periode,
        year,
        month,
        nutriscore_moyen_mois,
        nb_produits,
        ROUND(
            AVG(nutriscore_moyen_mois) OVER (
                ORDER BY year, month
                ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
            ),
            2
        ) AS moyenne_mobile_3_mois
    FROM mensuel
)
SELECT
    periode,
    nutriscore_moyen_mois,
    moyenne_mobile_3_mois,
    ABS(nutriscore_moyen_mois - moyenne_mobile_3_mois) AS ecart
FROM mensuel_mobile
WHERE moyenne_mobile_3_mois IS NOT NULL
ORDER BY ecart DESC
LIMIT 1;

```

DAG Airflow dans Docker

Pour automatiser l'exécution du pipeline NutriScan, j'ai utilisé Apache Airflow. C'est un outil d'orchestration qui permet de planifier, organiser et suivre l'exécution de plusieurs traitements dans un ordre précis. Il sert à lancer automatiquement les scripts PySpark de transformation des données, depuis la couche Silver jusqu'à la couche Gold, puis à exécuter les tests qualité à la fin.

L'intérêt principal d'Airflow est de ne plus exécuter les scripts manuellement un par un. Grâce au DAG, nous pouvons définir les dépendances entre les étapes : les scripts de staging doivent s'exécuter avant les dimensions, les dimensions doivent être créées avant la table de faits, et les tests doivent être lancés en dernier.

D'un point de vue configuration, il a fallu ajouter les dossiers contenant les scripts pyspark python dans le conteneur docker. Il a également fallu rajouter Java et les librairies python dans le conteneur, afin que l'exécution des scripts n'échouent pas.

Fonctionnement du DAG Airflow

Le pipeline est défini dans un fichier python nommé `nutriscan_pipeline.py`. Un DAG représente un graphe d'exécution : il contient plusieurs tâches et définit dans quel ordre elles doivent être lancées.

Dans notre cas, le DAG est planifié pour s'exécuter tous les jours à 12h00.

Exemple d'exécution du DAG :

List Dag Run

Search

+Actions

←

Record Count: 5

☐	State	Dag Id	Logical Date	Run Id	Run Type	Queued At	Start Date	End Date	Note	External Trigger	Conf	Duration
☐	success	nutriscan_pipeline	2026-07-01, 08:53:45	manual__2026-07-01T08:53:45.127457+00:00	manual	2026-07-01, 08:53:45	2026-07-01, 08:53:46	2026-07-01, 09:31:11	True		{}	37M:25s
☐	success	nutriscan_pipeline	2026-06-30, 12:07:14	manual__2026-06-30T12:07:14.647006+00:00	manual	2026-06-30, 12:07:14	2026-06-30, 12:07:15	2026-06-30, 12:48:08	True		{}	40M:52s
☐	success	nutriscan_pipeline	2026-06-30, 12:00:00	scheduled__2026-06-30T12:00:00+00:00	scheduled	2026-07-01, 12:00:01	2026-07-01, 12:00:01	2026-07-01, 12:31:07	False		{}	31M:5s
☐	success	nutriscan_pipeline	2026-06-30, 11:55:03	manual__2026-06-30T11:55:03.750853+00:00	manual	2026-06-30, 11:55:03	2026-06-30, 11:55:42	2026-06-30, 11:55:42	True		{}	<1s
☐	success	nutriscan_pipeline	2026-06-29, 12:00:00	scheduled__2026-06-29T12:00:00+00:00	scheduled	2026-06-30, 13:23:19	2026-06-30, 13:23:19	2026-06-30, 13:56:03	False		{}	32M:44s

Description des tâches du DAG

Le DAG est organisé en trois grandes parties.

1. Les tâches de staging

Les premières tâches exécutent les scripts de la couche Silver :

- `stg_products`

- stg_nutrition
- stg_countries
- stg_brands
- stg_date
- stg_category

Chaque tâche utilise un BashOperator, qui permet de lancer une commande shell dans le conteneur Airflow. Exemple de BashOperator :

```
t_stg_products = BashOperator(
    task_id='stg_products',
    bash_command='python /opt/airflow/transform/staging/stg_products.py'
)
```

Cette commande exécute le script PySpark chargé de créer la table de staging des produits. Ces tâches peuvent s'exécuter en parallèle, car elles lisent toutes la couche RAW et produisent chacune une table Silver différente.

2. Les tâches de dimensions Gold

Une fois les données staging générées, le DAG lance les scripts de création des dimensions :

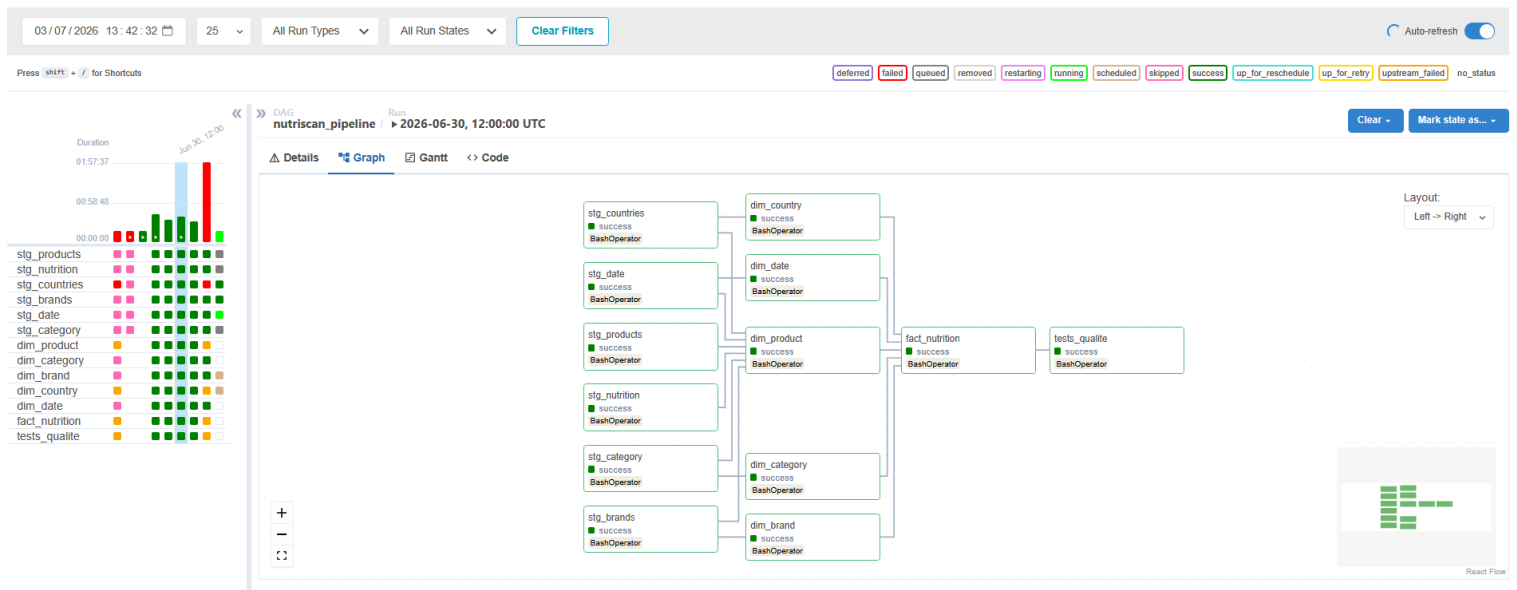
- dim_product
- dim_category
- dim_brand
- dim_country
- dim_date

Chaque dimension dépend de sa table staging correspondante. Par exemple :

```
t_stg_category >> t_dim_category
```

Cela signifie que la dimension catégorie ne peut être construite qu'après la création de stg_category.

Visualisation des dépendances et ordonnancement des tâches :



3. La table de faits et les tests

La tâche fact_nutrition est exécutée après la création de toutes les dimensions :

```
[t_dim_product, t_dim_category, t_dim_brand, t_dim_country, t_dim_date] >>
t_fact_nutrition
```

Cette tâche construit la table de faits finale en reliant les mesures nutritionnelles aux dimensions.

Enfin, les tests qualité sont lancés en dernier :

```
t_fact_nutrition >> t_tests
```

Cela garantit que les contrôles sont effectués uniquement après la génération complète de la couche Gold.

Script python nutriscan_pipeline.py :

```
# dags/nutriscan_pipeline.py
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'tom',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    dag_id='nutriscan_pipeline',
    default_args=default_args,
```

```

schedule='0 12 * * *', # tous les jours a 12h du matin (Airflow 2.4+)
start_date=datetime(2026, 6, 1),
catchup=False,
) as dag:

# Staging
t_stg_products = BashOperator(task_id='stg_products',
    bash_command='python /opt/airflow/transform/staging/stg_products.py')
t_stg_nutrition = BashOperator(task_id='stg_nutrition',
    bash_command='python /opt/airflow/transform/staging/stg_nutrition.py')
t_stg_countries = BashOperator(task_id='stg_countries',
    bash_command='python /opt/airflow/transform/staging/stg_countries.py')
t_stg_brands = BashOperator(task_id='stg_brands',
    bash_command='python /opt/airflow/transform/staging/stg_brands.py')
t_stg_date = BashOperator(task_id='stg_date',
    bash_command='python /opt/airflow/transform/staging/stg_date.py')
t_stg_category = BashOperator(task_id='stg_category',
    bash_command='python /opt/airflow/transform/staging/stg_category.py')

# Marts
t_dim_product = BashOperator(task_id='dim_product',
    bash_command='python /opt/airflow/transform/marts/dim_product.py')
# A COMPLETER : t_dim_category, t_dim_brand, t_dim_country, t_dim_date
t_dim_category = BashOperator(task_id='dim_category',
    bash_command='python /opt/airflow/transform/marts/dim_category.py')
t_dim_brand = BashOperator(task_id='dim_brand',
    bash_command='python /opt/airflow/transform/marts/dim_brand.py')
t_dim_country = BashOperator(task_id='dim_country',
    bash_command='python /opt/airflow/transform/marts/dim_country.py')
t_dim_date = BashOperator(task_id='dim_date',
    bash_command='python /opt/airflow/transform/marts/dim_date.py')

# A COMPLETER : t_fact_nutrition
t_fact_nutrition = BashOperator(task_id='fact_nutrition',
    bash_command='python /opt/airflow/transform/marts/fact_nutrition.py')

# Tests
t_tests = BashOperator(task_id='tests_qualite',
    bash_command='python
/opt/airflow/transform/tests/test_dim_product.py')

# Dependances
[t_stg_products, t_stg_nutrition, t_stg_countries, t_stg_brands,
t_stg_date, t_stg_category] >> t_dim_product
t_stg_category >> t_dim_category
t_stg_brands >> t_dim_brand
t_stg_countries >> t_dim_country
t_stg_date >> t_dim_date

```

```
[t_dim_product, t_dim_category, t_dim_brand, t_dim_country, t_dim_date] >>
t_fact_nutrition

# les tests doivent s'exécuter EN DERNIER
t_fact_nutrition >> t_tests
```

Gestion des dépendances

Les dépendances permettent à Airflow de s'assurer que les étapes s'exécutent dans le bon ordre. Dans le pipeline :

- Les scripts de staging s'exécutent d'abord ;
- Les dimensions sont générées à partir des tables Silver ;
- La table fact_nutrition est construite après toutes les dimensions ;
- Les tests qualité sont exécutés en dernier.

Cette organisation permet d'éviter qu'une table Gold soit créée à partir de données incomplètes ou non encore générées.

Airflow permet donc d'automatiser et de fiabiliser l'ensemble du pipeline NutriScan. Au lieu de lancer manuellement chaque script PySpark, il y a un DAG qui orchestre les traitements dans le bon ordre. Cette orchestration facilite les relances, permet de suivre les erreurs dans l'interface web, et prépare le projet à une exécution régulière et automatisée.

Rapport data quality final

Analyse de la qualité des données sources. Anomalies identifiées et quantifiées.
Stratégie de traitement retenu et justifiée.

1. Taux de complétude

Dimension dim_product :

Champ	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
nutriscore_grade	4 521 540	1 364 541	3 156 999	30,20 %
quantity	4 521 540	1 429 210	3 092 330	31,60 %
environmental_score_grade	4 521 540	3 685 620	835 920	81,50 %
product_name	4 521 540	4 243 549	277 991	93,90 %
product_sk	4 521 540	4 521 540	0	100,00 %

Dimension dim_brand :

Champ	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
brand_tag	348 775	348 761	14	100,00 %
brand_sk	348 775	348 775	0	100,00 %

Dimension dim_country :

Champ	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
country_tag	384	384	0	100,00 %
country_sk	384	384	0	100,00 %

Dimension dim_date :

Champ	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
date_sk	3 856	3 856	0	100,00 %
full_date	3 856	3 856	0	100,00 %

Table de faits fact_nutrition :

Champ	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
fiber_g	4 770 925	1 888 579	2 882 346	39,60 %
category_sk	4 770 925	2 056 034	2 714 891	43,10 %
salt_g	4 770 925	3 054 807	1 716 118	64,00 %

brand_sk	4 770 925	3 079 613	1 691 312	64,50 %
saturated_fat_g	4 770 925	3 232 494	1 538 431	67,80 %
sugars_g	4 770 925	3 294 387	1 476 538	69,10 %
fat_g	4 770 925	3 470 882	1 300 043	72,80 %
proteins_g	4 770 925	3 475 616	1 295 309	72,80 %
energy_kcal	4 770 925	3 499 459	1 271 466	73,30 %
date_sk	4 770 925	4 770 925	0	100,00 %
nutrition_fact_sk	4 770 925	4 770 925	0	100,00 %

2. Taux de doublons

Unicité des clés SK :

Contrôle	Nombre de doublons
dim_country.country_sk	0
dim_date.date_sk	0
dim_category.category_sk	0
dim_brand.brand_sk	0
dim_product.product_sk	0

Unicité des clés métier :

Contrôle	Nombre de doublons
dim_date.full_date	0
dim_country.country_tag	0
dim_category.category_tag	0
dim_brand.brand_name	0
dim_product.product_code	0

Grain de la table de fait respecté :

Contrôle	Résultat
Doublons sur le couple product_sk + country_sk	0

Correspondance entre la table de fait et les dimensions :

Anomalie	Volume
category_sk sans correspondance dim_category	0
date_sk sans correspondance dim_date	0
country_sk sans correspondance dim_country	0
brand_sk sans correspondance dim_brand	0
product_sk sans correspondance dim_product	0

Valeurs nutritionnelles aberrantes :

Valeur nutritionnelle	Volume
Nombre total de lignes	4 770 925
energy_kcal aberrantes	20 216
fat_g aberrantes	6 269
saturated_fat_g aberrantes	1 042
sugars_g aberrantes	6 518
salt_g aberrantes	2 499
fiber_g aberrantes	616
proteins_g aberrantes	1 770
saturated_fat_g supérieur à fat_g	3 695
Lignes avec au moins une anomalie	29 122

Contrôle du Nutri-Score :

Nombre lignes avec Nutri-Score renseigné	Nutri-Score aberrants
1 538 741	2 944

Grades Nutri-Score invalides dans dim_product :

Nombre total produits	Grades invalides
------------------------------	-------------------------

4 521 540	0
-----------	---

Globalement, le rapport de data quality montre que la couche Gold est cohérente et exploitable pour l'analyse. Les points les plus importants sont validés : les clés techniques des dimensions sont uniques, les clés métier principales ne présentent pas de doublons, l'intégrité référentielle entre la table de faits et les dimensions est respectée, et le grain de fact_nutrition est bien maîtrisé avec une ligne par produit et par pays. Cela confirme que le modèle dimensionnel est propre et utilisable dans Trino et Superset.

Cependant certaines limites restent présentes, principalement à cause de la qualité initiale des données Open Food Facts. Plusieurs champs restent incomplets, notamment category_sk, brand_sk, fiber_g ou encore certains scores nutritionnels. On observe aussi encore quelques valeurs nutritionnelles aberrantes, comme des calories supérieures à 900 kcal ou des nutriments supérieurs à 100 g. Ces anomalies ne remettent pas en cause la structure du modèle, mais elles montrent que certaines analyses devront être interprétées avec prudence, surtout lorsqu'elles reposent sur des champs peu complétés.

La qualité finale est satisfaisante : la structure Gold est stable, les jointures fonctionnent correctement, les dimensions sont bien dédupliquées et la table de faits respecte le grain attendu. Les problèmes restants sont surtout liés aux données sources, et non au pipeline lui-même. Le modèle peut donc être utilisé pour les requêtes OLAP et les dashboards, tout en gardant en tête les limites de complétude et les quelques valeurs aberrantes restantes.

Conclusion

Nous avons structuré le pipeline en trois couches : Raw, Silver et Gold. Les données brutes sont conservées, puis nettoyées et normalisées avant d'être transformées en modèle dimensionnel exploitable dans Trino et Superset.

Le modèle final repose sur des dimensions produit, marque, catégorie, pays et date, ainsi qu'une table de faits fact_nutrition au grain produit × pays. Cela nous permet d'analyser les données nutritionnelles selon plusieurs axes : produit, marque, catégorie, pays et période.

Nous avons aussi automatisé le pipeline avec Airflow et ajouté des tests qualité pour vérifier la cohérence des tables Gold. Le rapport qualité final montre que la structure est

globalement fiable : les clés sont uniques, les jointures fonctionnent et le grain de la table de faits est respecté.

Quelques limites restent présentes, notamment des champs incomplets et certaines valeurs nutritionnelles aberrantes, principalement liées à la qualité des données sources. Malgré cela, l'architecture est cohérente, exploitable et prête pour les analyses OLAP et les dashboards Superset.